MAILDESK — EMAIL MANAGEMENT SYSTEM

Integrated OOPS and DBMS Project Report

Submitted by

_____ – _____

_____ – _____

_____ – _____

of

BACHELOR OF ENGINEERING

in

Department Name: _____

INSTITUTION: _____

(Institution / autonomous status, if applicable)

[INSTITUTION LOGO]

November 2026

BONAFIDE CERTIFICATE

This is to certify that the project report titled “MAILDESK — EMAIL MANAGEMENT SYSTEM” is the bonafide work of the students whose names and registration numbers are entered below. The work was completed as an academic project integrating Object Oriented Programming in Java and Database Management Systems.

The report describes the design, implementation, database structure, interface, and observed outputs of the submitted application. The student team has prepared the work for academic assessment and has identified the implemented functions and the known limitations of the project.

STUDENT DETAILS AND SIGNATURES

NAME OF STUDENT	REGISTER NUMBER	SIGNATURE
_____	_____	_____
_____	_____	_____
_____	_____	_____

INTERNAL FACULTY

Ms.DHANALAKSHMI K
Assistant professor
Object oriented programming in java
Artificial Intelligence and Data Science

Ms.DHANALAKSHMI K
Assistant professor
Database management systems
Artificial Intelligence and Data Science

Date of submission / practical examination: _____

INTERNAL EXAMINER	EXTERNAL EXAMINER
Signature: _____	Signature: _____
Date: _____	Date: _____

ABSTRACT

Maildesk is a Java desktop email-management system that brings common Gmail tasks into one focused workspace. A user can connect through Google OAuth, browse Inbox, Sent, and Gmail Spam, search the locally cached sender, recipient, subject, and body, read messages, and compose plain-text email. The Swing application uses an application service boundary so the interface, Gmail integration, local spam analysis, and persistence can be understood as separate parts.

The project's distinctive feature is an explainable second opinion for suspicious messages. A deterministic Java rule classifier assigns a score from visible signals such as blocked senders, suspicious phrases, many links, and urgent subject wording. The interface presents reasons with the score so the mailbox owner can review and correct the decision. A score is a rule score, not a probability. Gmail's own Spam label remains distinct from Maildesk's locally suspected messages. A saved user correction is retained per account.

For predictable demonstrations, the live connection and refresh retrieve up to six recent messages from each of Inbox, Sent, and Gmail Spam, deduplicate message IDs, and cap the local sample at 18 unique messages. The application stores accounts, messages, recipients, labels, spam results, feedback, and blocked senders in MySQL through JDBC. Composite account/message keys scope cached records to an account. OAuth tokens and database passwords remain in local configuration and are not stored in the message tables.

The separate demo launcher uses fictional in-memory messages. It does not connect to Gmail or MySQL, use the network, or deliver real email. The project uses Java 17 or newer, Swing, Maven, the Gmail REST API, Google OAuth, and MySQL 8. The report documents its flow, use cases, data model, modules, source-level implementation, screenshots, setup, and verification coverage. Attachments, rich text, scheduled synchronization, broad public OAuth distribution, and machine-learning classification are future work, not current features.

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE NO.
1	INTRODUCTION: problem statement, project overview, purpose, scope	5–7
2	FUNCTIONALITIES PROVIDED	8
3	SYSTEM FLOW DIAGRAM: dataflow, use cases, ER model	9–12
4	MODULE DESCRIPTION: workspace, authorization, mailbox, messages, spam, demo	13–15
5	IMPLEMENTATION: Java contracts, Gmail, spam rules, JDBC, setup, verification	16–24
6	CONCLUSION	25
7	REFERENCES	26
A	SELECTED JAVA CLASSES AND DATABASE SCHEMA	27–35
B	APPLICATION OUTPUT AND SCREENSHOTS	36–40

Student names, register numbers, department, institution name and logo, and submission / practical-examination date are left for completion. Faculty details are shown on the bonafide certificate. No personal or institutional details from the supplied example have been copied into this report.

1 - INTRODUCTION

1.1 PROBLEM STATEMENT

Email users often move between separate mailbox views to find recent work, inspect a message, compose a reply, and check suspicious requests. A basic spam filter can hide a message or provide little explanation for its decision. When users cannot see the reason, they have less confidence in the result and may have difficulty correcting a false positive.

A student-scale email management application should combine the common tasks in a clear interface while keeping the user involved in security decisions. It also needs a reliable way to preserve a small mailbox cache in a relational database without turning the project into a full Gmail mirror.

Maildesk addresses this problem with a focused desktop workspace, a bounded recent-message sample, MySQL persistence, and an explainable local spam review. The user sees matched warning reasons and can mark a received message as spam or not spam. Gmail's own Spam folder and Maildesk's local suggestions remain separate views.

The difficulty is not simply that a mailbox contains many messages. The same mailbox combines routine notices, personal correspondence, receipts, work requests, and potentially deceptive messages. A user must locate relevant mail while deciding which content deserves caution. A useful student project should therefore demonstrate both ordinary message handling and a transparent review mechanism. It should also explain exactly which data is local, which action reaches Gmail, and how the user can correct a recommendation.

1 - INTRODUCTION

1.2 PROJECT OVERVIEW

Maildesk is a Java Swing desktop client connected to Gmail through Google's OAuth flow and Gmail REST API. Its main window presents a navigation rail, mailbox list, search controls, a message reader pane, and a compose form. JDBC connects its application cache to a local MySQL database.

The core service is exposed through an EmailService interface. GmailEmailService coordinates remote Gmail actions with the local store; DemoEmailService implements the same interface using fictional in-memory messages. Because both implementations share a contract, the interface can be demonstrated without credentials, network access, or a running database.

On live startup, the system requests six recent items from Inbox, Sent, and Spam, deduplicates IDs, then stores no more than 18 unique messages. Local spam analysis attaches a score and reasons to received messages. Search covers the cached sample. Compose sends plain-text email after validation. The database persists message data, Gmail labels, spam explanations, feedback, and blocked senders per account.

The application has two deliberate execution paths. The live path uses the user-authorized Gmail account and MySQL. The presentation path uses deterministic fictional content held only in memory. Both paths use the same EmailService interface, which keeps the UI behavior consistent while making the demo safe and independent of internet access. The report treats these paths separately wherever a screenshot or test could otherwise be mistaken for a live mailbox action.

1 - INTRODUCTION

1.3 PURPOSE, OBJECTIVES, AND SCOPE

Maildesk demonstrates an end-to-end Java and DBMS application that centralizes reading, searching, composing, and suspicious-message review. Its purpose is to show how an object-oriented service boundary connects a Swing interface to Gmail and a relational database while keeping a person involved in security decisions.

Project objectives

- Use Google OAuth without collecting a Gmail password; browse Inbox, Sent, Gmail Spam, and local Suspected Spam.
- Search the bounded sample, read messages, validate plain-text email, and send only after explicit user action.
- Persist account-scoped messages, recipients, labels, spam reasons, and user corrections through JDBC.
- Explain rule matches and permit correction; provide an offline fictional demo that uses no network or database.

INCLUDED IN THIS PROJECT	OUTSIDE THIS BUILD
Gmail Desktop OAuth; bounded 18-message cache; local MySQL persistence.	Full Gmail mirror, hosted multi-user service, or public administration.
Plain-text read, search, compose, refresh, and spam review.	Attachments, rich text, scheduled sync, or multi-account UI.
Deterministic explainable spam rules and account-specific feedback.	Trained machine-learning model or probability-calibrated score.

Live use requires local MySQL and a user-provided Google Desktop OAuth client. The project’s test-mode consent configuration is suitable for the student demonstration, not broad public distribution.

2 - FUNCTIONALITIES PROVIDED

Maildesk offers a compact set of workflows that can be explained and demonstrated from one workspace. A live mailbox owner signs in through Google, then chooses one of the four mailbox views. The interface updates the list and reader pane without replacing the connected account.

FUNCTION	USER VALUE	BOUNDARY
Connect Gmail	Use Google sign-in and load recent mail.	Requires configured Desktop OAuth credentials and consent.
Inbox / Sent / Gmail Spam	Read common mailbox categories.	Gmail labels identify provider folders.
Suspected Spam	Review the local rule model’s suggestions.	Separate from Gmail’s Spam folder.
Search	Find a message in the cached sample.	Does not search every remote Gmail message.
Compose	Send a validated plain-text message.	No attachments or rich formatting.
Correct spam	Save the user’s decision and update Gmail.	Only explicit user actions change Gmail labels.
Offline demo	Explore fictional data without external setup.	No network, MySQL, or real mail delivery.

Refresh replaces the local recent snapshot; it does not delete Gmail messages. Failed reads do not become an empty successful result. A local correction is associated with the account and may override the later automatic rule result.

3 - SYSTEM FLOW DIAGRAM

3.1 OVERALL SYSTEM FLOW

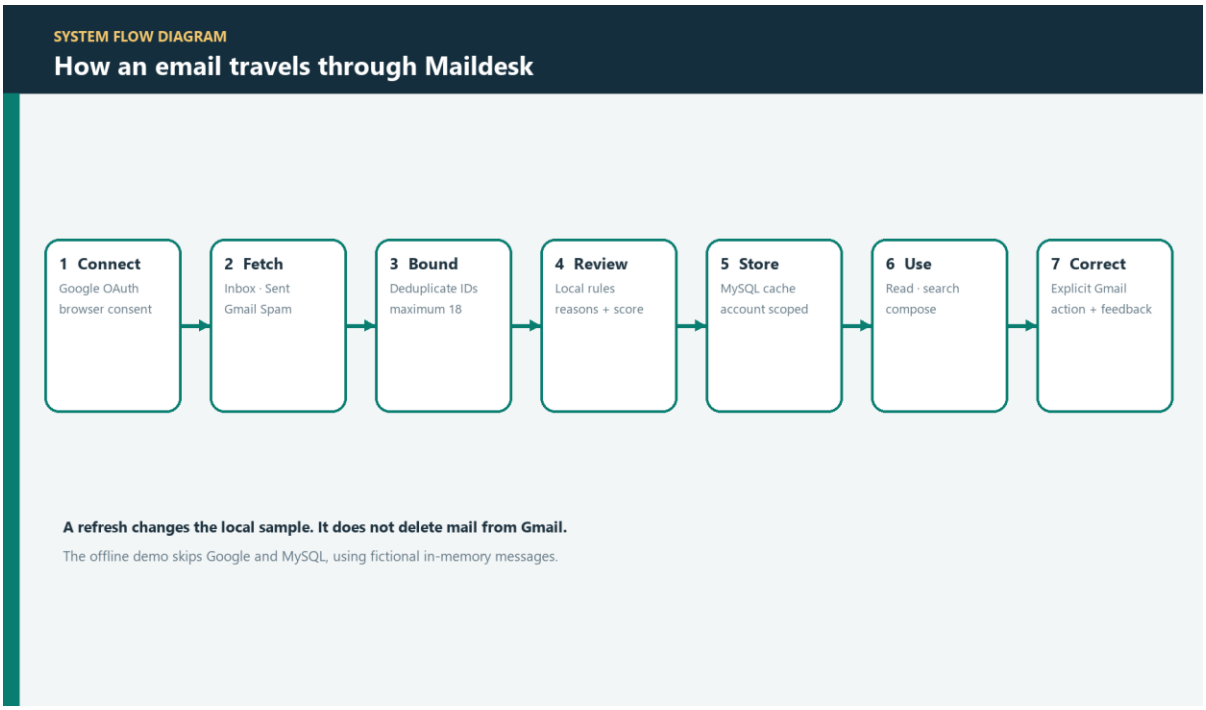


Figure 1: System flow from account connection through user review

The flow starts when a user connects Gmail. Maildesk requests up to six recent messages from each supported provider label. IDs returned from more than one label are deduplicated before the 18-item cap is applied. The service evaluates received messages locally and writes the snapshot and analysis into MySQL.

The person then reads, searches, or composes. Gmail state changes occur only after a corresponding user action. A spam correction is also saved to local feedback. In offline demo mode, the connection step is simulated and the data path remains in memory.

Each boundary has a defined responsibility. OAuth establishes identity and permission; Gmail supplies provider messages and receives explicit user changes; the classifier adds a local review signal; the JDBC store persists a bounded cache and feedback; the UI presents the combined state. The demo mode replaces the live integrations with in-memory behavior while preserving the workflow. A failure at one boundary is surfaced as a failed action rather than being silently converted into successful empty data.

3 - SYSTEM FLOW DIAGRAM

3.2 DATAFLOW DIAGRAM

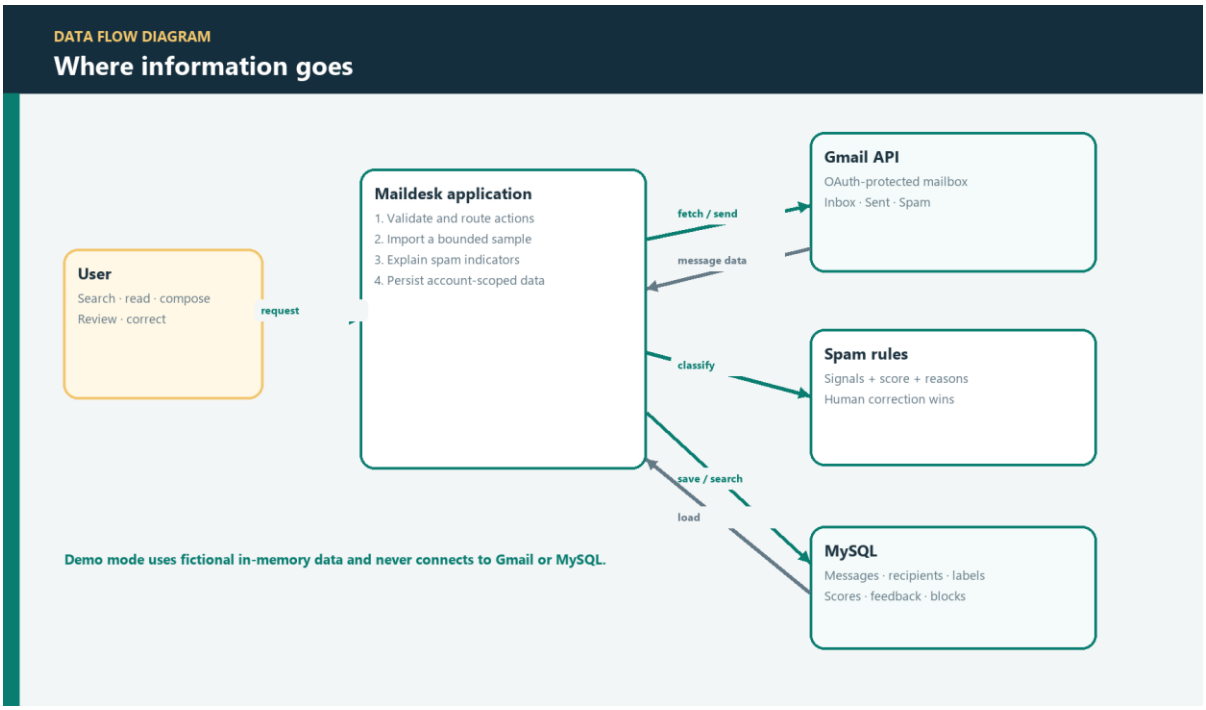


Figure 2: Data flow between the person, application, Gmail, classifier, and MySQL

The Swing interface accepts searches and explicit actions. The application service validates the selected mailbox or draft and routes requests to the Gmail gateway or the database store. Gmail returns message metadata and body text; the local classifier derives a score and explanation from the message fields and account-specific blocked-sender list.

The database stores a cache and the results needed for the local UI. OAuth tokens remain in a local protected token store rather than in the email tables. The demo implementation bypasses all external boxes and reads fictional fixtures from memory.

The data flow distinguishes message content from authorization credentials. Message metadata and plain-text content pass through the service into the local cache; OAuth tokens are kept in a separate local token directory. Search is limited to the loaded sample. The database does not act as a public server and is not exposed to other accounts. This separation helps explain why neither a message's Gmail password nor an OAuth secret is stored in the relational message rows.

3 - SYSTEM FLOW DIAGRAM

3.3 USE CASE DIAGRAM

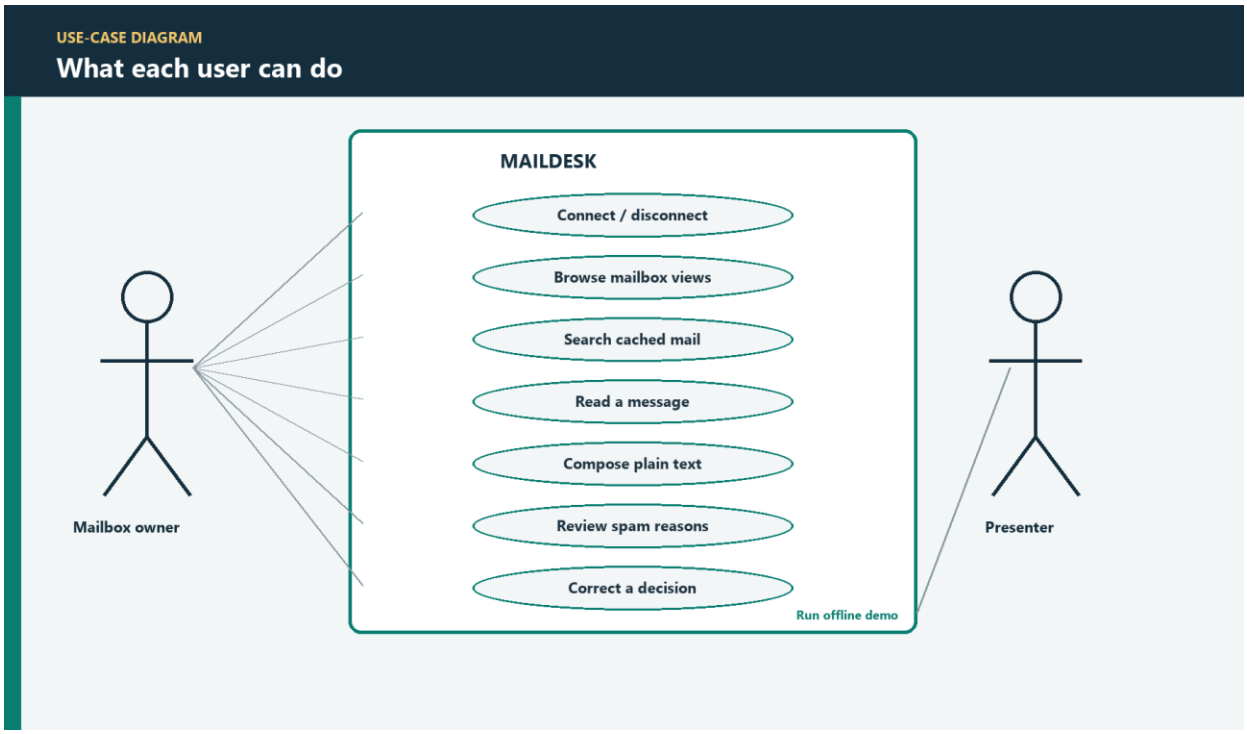


Figure 3: Mailbox owner use cases and the presenter's demo entry

ACTOR	USE CASE	DESCRIPTION
Mailbox owner	Connect / disconnect	Authorize a Gmail account or end the local session.
Mailbox owner	Browse / search	Choose Inbox, Sent, Spam, or Suspected Spam; search the cache.
Mailbox owner	Read / compose	Open a cached message or compose a validated plain-text message.
Mailbox owner	Review / correct	Read rule reasons and mark a received message spam or not spam.
Presenter	Run demo	Explore fictional content without Gmail or database setup.

The primary actor is the mailbox owner because every live action is performed under that person's authorization. The presenter actor has a narrower role: start offline demo mode and show fictional output. Use cases that change Gmail—sending, marking spam, clearing spam, and marking read—are distinguished from local actions such as opening a view or searching the current sample. This makes the user's authorization and intent visible in the requirements.

3 - SYSTEM FLOW DIAGRAM

3.4 ENTITY RELATIONSHIP DIAGRAM

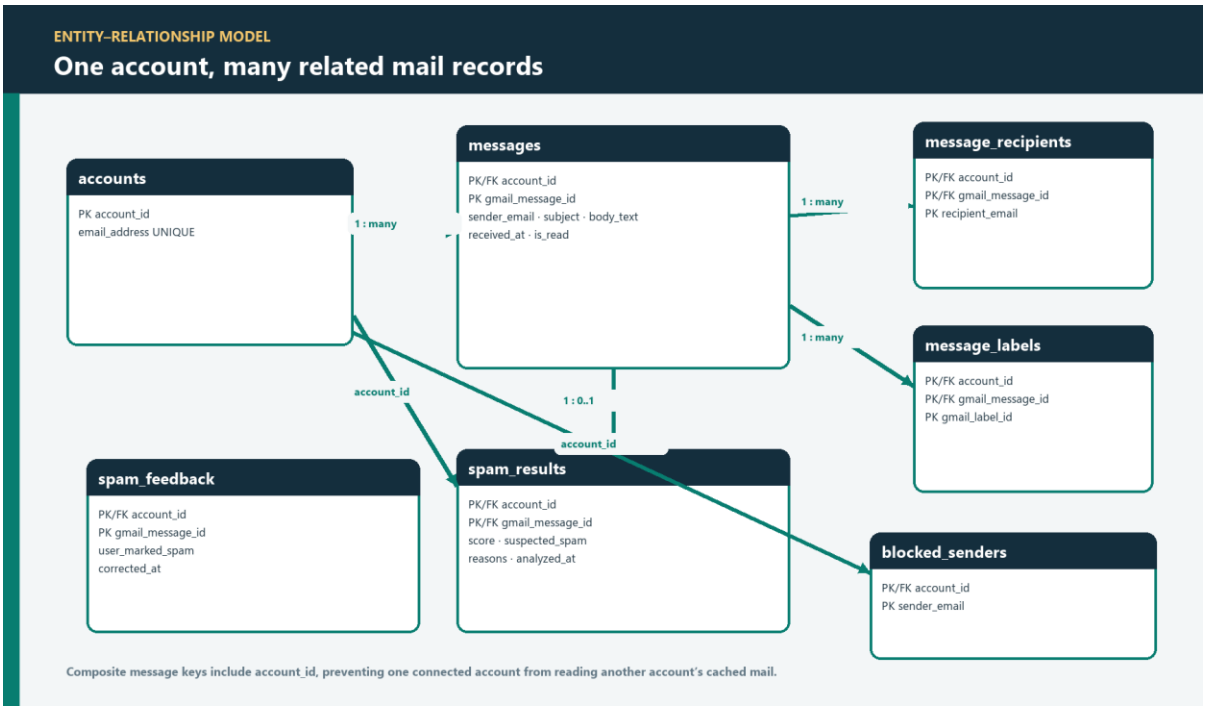


Figure 4: MySQL entities and account-scoped key relationships

The accounts table identifies each connected mailbox. Messages use the composite key (account_id, gmail_message_id). Recipient and label rows reference that key. Spam results are stored per message, while feedback and blocked-sender rows are associated with the account. This structure keeps repeated recipients and Gmail labels out of a single overloaded message column.

Foreign keys enforce parent-child relationships. The composite message key ensures lookups and writes are scoped to the active account. sql/schema.sql contains the full table definitions and constraints.

The schema resolves one-to-many relationships explicitly. One account owns many messages; one message can have several recipients and Gmail labels; one message can have a current spam analysis and a user feedback record. Composite keys and foreign keys provide referential integrity, while account IDs prevent a lookup from treating two users' identical Gmail message IDs as the same local row. The complete DDL is reproduced in Appendix A.

4 - MODULE DESCRIPTION

4.1 ACCOUNT ACCESS, MAILBOXES, AND RECENT MAIL

Maildesk presents a welcome view, mailbox workspace, reader, compose panel, spam-review view, application service, Gmail adapter, and JDBC store. The Swing interface depends on the EmailService contract; provider and database details remain in integration classes. This separation also allows the offline demo to use the same screens without real account access.

Account access

The live Connect action opens Google in the system browser and uses a Desktop OAuth client with PKCE. Maildesk does not ask for or store a Gmail password. The callback is bound to 127.0.0.1, validates the returned state, and exchanges the authorization code. Each user must configure the Gmail API, OAuth consent screen, test-user access, and local token directory.

Mailbox navigation and sample

Inbox, Sent, and Gmail Spam correspond to provider labels. Suspected Spam is a separate local review view. The central table shows sender or recipient, subject, and time; the selected message opens in a reader pane. Connection and refresh request up to six recent messages from each provider label, deduplicate IDs, and retain at most 18 unique messages. Search is limited to this local sample, not all of Gmail.

4 - MODULE DESCRIPTION

4.2 SEARCH, READING, AND COMPOSITION

Search and reader

Search checks sender, recipients, subject, and cached plain-text body. An empty query restores the selected mailbox. The reader displays sender, recipient, subject, date, body, provider labels, and any spam explanation. Plain text is shown as text, not interpreted as HTML. Missing message content does not prevent the remaining fields from appearing.

Compose and send

Compose collects one recipient, subject, and plain-text body. Validation rejects an invalid address, blank or multi-line subject, or empty body before a network request; the service validates again. The Gmail adapter constructs a UTF-8 MIME message. The user must explicitly press Send, and the interface disables duplicate submission while the operation is pending.

A successful Gmail send is reported even if saving its local cache copy subsequently fails. Sends are not automatically retried after ambiguous network errors because the first request may already have reached Gmail. Attachments, rich text, multiple recipients, and synchronized drafts are outside this version.

4 - MODULE DESCRIPTION

4.3 SPAM REVIEW, CORRECTION, AND DEMONSTRATION

SpamClassifier is a transparent deterministic second opinion. A blocked sender adds 50 points; each matched suspicious phrase adds 20, up to three; five or more links add 15; and urgent subject wording adds 10. A score of 50 reaches the configured review threshold. The score is not a probability or machine-learning prediction; the view displays the matched reasons so the mailbox owner can decide.

Gmail’s Spam folder is distinct from Maildesk’s locally suspected messages. Mark as spam / Not spam makes an explicit Gmail label request and saves a separate account-scoped feedback record. If Gmail cannot confirm the action, the UI reports the failure instead of claiming success.

The offline demo uses fictional messages held in memory. It supports navigation, search, read, compose simulation, and correction without network, Gmail, or MySQL. The visible demo notice clarifies that no real email is sent. Appendix B contains the five captured application outputs; addresses and messages are fictional.

5 - IMPLEMENTATION

5.1 SOURCE ORGANIZATION AND OBJECT-ORIENTED DESIGN

SOURCE AREA	RESPONSIBILITY
ems.ui	Swing frames, panels, table model, compose form, visual theme.
ems.service	EmailService contract and live workflow coordination.
ems.gmail	OAuth, token handling, Gmail REST, MIME, and parsing.
ems.storage / ems.model	JDBC persistence, schema setup, and domain values.
spam, demo, tests	Explainable rules, offline service, and verification sources.

The project divides interface, business workflow, provider access, spam analysis, and persistence into separate responsibilities. Account, EmailMessage, Mailbox, and SpamResult carry domain data rather than exposing HTTP payloads or JDBC rows to the UI. EmailMessage copies its collection fields so callers cannot mutate a saved snapshot through an earlier list.

The complete report appendix includes the focused source classes that explain the distinguishing behavior, plus the complete database schema. The repository itself contains every production class and test.

5 - IMPLEMENTATION

5.2 SERVICE CONTRACT AND SWING RESPONSIVENESS

EmailService defines the operation names shared by the live Gmail implementation and the in-memory demo: connect, disconnect, refresh, list messages, open a message, send, read a spam result, and correct spam classification. This contract lets the UI use one workflow without knowing which data source is active.

SwingWorker moves network and JDBC operations away from the event-dispatch thread. Its background phase calls the service; its completion phase updates the table, reader, status text, and controls on the UI thread. Busy-state handling prevents duplicate requests and restores the controls on both success and failure.

Application exceptions carry user-actionable messages. The UI does not expose provider tokens or database passwords, and a failed refresh preserves the previous state instead of presenting an error as a valid empty inbox.

5 - IMPLEMENTATION

5.3 GOOGLE OAUTH AND GMAIL OPERATIONS

GmailAuth implements the native authorization-code flow with PKCE. It creates a random state and verifier, derives an S256 challenge, starts an ephemeral loopback callback, validates state, and exchanges the returned code over HTTPS. The temporary server is closed after success, cancellation, or timeout. Tokens are held in a local token directory separate from message rows.

GmailApiClient and GmailGateway isolate request construction, access-token use, HTTP status handling, parsing, and provider-specific operations. The service reads the account profile, lists messages by label, fetches details, submits MIME mail, and applies explicit label changes.

OPERATION	MAILDESK BEHAVIOR
Profile / list	Identify the account and request bounded label results.
Get message	Read metadata, plain-text body, timestamp, and labels.
Send	Submit a user-composed plain-text MIME message.
Modify	Change read or Spam labels only after a user action.

Read requests use bounded retry for transient failures. Sends and label changes are not replayed automatically after ambiguous results.

5 - IMPLEMENTATION

5.4 BOUNDED IMPORT AND EXPLAINABLE SPAM CLASSIFICATION

Connection and refresh fetch at most six recent entries from Inbox, Sent, and Gmail Spam. The client deduplicates Gmail IDs shared across labels; GmailEmailService applies the final 18-message cap before the store replaces the local snapshot. A refresh changes Maildesk's cache and never deletes Gmail messages. Transactional replacement avoids presenting a partial sample as complete.

The deterministic classifier checks an account-specific blocked-sender list, suspicious phrases, message link count, and urgency language in the subject. It returns a score and matched reasons. The threshold is a review rule rather than a calibrated probability. User feedback remains separate from the automatic score and can override the displayed suggestion for that account and message.

The SpamClassifier and SpamRules source listings in Appendix A show the exact rule weights and decision flow used by the application.

5 - IMPLEMENTATION

5.5 DATABASE DESIGN: ACCOUNTS, MESSAGES, AND RELATIONSHIPS

Accounts use a generated local key and unique email address. A message is identified by the composite key (account_id, gmail_message_id); this avoids collisions between accounts and supports idempotent refresh. Sender, subject, body text, timestamp, and read state are stored in the message row. The schema uses utf8mb4 for international content.

Recipients and Gmail labels are stored in separate child tables because one message may contain multiple values of each type. Their composite foreign keys point to the exact parent account and message. Cascading deletion removes child cache rows with a parent; it has no effect on Gmail.

Spam results, feedback, and blocked senders are separate account-scoped records. The result stores the latest automatic score and reasons, feedback stores the user's later decision, and the blocked-sender table supplies a classifier input. The full CREATE TABLE schema is reproduced in Appendix A.

5 - IMPLEMENTATION

5.6 JDBC TRANSACTIONS AND PERSISTENCE

JdbcMailStore implements the MailStore contract with prepared statements. It resolves the local account, verifies message ownership, upserts message values, replaces recipient and label rows, recalculates spam explanations, and maintains the bounded sample. User values are bound as parameters rather than concatenated into SQL strings.

Snapshot replacement and related writes are grouped in a transaction. The store disables auto-commit, applies the full snapshot, and commits only after all operations succeed. A SQL or runtime failure triggers rollback and is returned to the service. This prevents partial cache state from appearing complete.

Appendix A includes the full database DDL and selected source methods that demonstrate account checks, snapshot replacement, and rollback behavior.

5 - IMPLEMENTATION

5.7 CONFIGURATION AND DATA PROTECTION

AppConfig reads local properties and environment overrides for the database URL, username, password, OAuth client file, and token directory. The checked-in example contains no secret. Real OAuth JSON, token state, database passwords, and private mailbox exports are excluded from Git and are not included in this report's code listings.

DATA	LOCAL LOCATION	HANDLING
Database login	Properties or environment	Keep private; do not commit.
OAuth client JSON	User-selected file	Supply locally; not packaged.
OAuth tokens	Local token directory	Separate from message rows.
Cached messages	MySQL project database	Bounded and account-scoped.

A dedicated MySQL account with access only to the project database is recommended. Gmail sign-in uses Google's browser consent and the Gmail modify scope; Maildesk does not store the mailbox password.

5 - IMPLEMENTATION

5.8 ERROR HANDLING AND WINDOWS PACKAGE

The UI distinguishes missing OAuth setup, denied authorization, invalid user input, provider failure, and MySQL unavailability. Status text gives a next action without exposing tokens, passwords, or raw provider payloads. A failed message open clears stale reader details; a failed refresh retains the prior cached sample.

The Maven Wrapper builds the Java application. The production and demo launchers are separate. The Windows packaging script creates an x64 app image with a bundled runtime and a shareable ZIP. The package excludes OAuth credentials, saved tokens, and database passwords; each live installation requires local MySQL and user-provided OAuth configuration.

SwingWorker keeps slow API and database work off the event thread. Reads can be retried within a bounded policy; sends and label mutations are not automatically replayed.

5 - IMPLEMENTATION

5.9 VERIFICATION, LIMITATIONS, AND FUTURE WORK

AREA	CURRENT VERIFICATION / LIMIT
OAuth / Gmail	Local HTTP fixtures; live acceptance needs a configured test account.
Spam review	Rule weights, explanations, threshold, and user feedback.
Database	Schema, account isolation, and transactional snapshot behavior.
UI	Search, navigation, validation, busy state, and error display.
Scope	18 recent messages; one account; plain-text compose; manual refresh.

Automated verification does not send real email. A live walkthrough separately configures Google OAuth and MySQL, connects a test account, refreshes the sample, checks the row count, opens a message, and verifies account-scoped records in MySQL Workbench.

Future work may add pagination, attachments, multiple accounts, scheduled refresh, and measured classifier improvements. The current project does not claim a trained model or complete mailbox synchronization. Any future spam model should retain visible evidence and leave the final decision to the mailbox owner.

6 - CONCLUSION

Maildesk combines Java Swing, object-oriented service boundaries, Gmail OAuth and REST access, MySQL, and JDBC in one email-management application. The workspace supports mailbox navigation, local search, message reading, validated plain-text composition, and explicit label actions.

The report's principal project contribution is an explainable second opinion for suspicious messages. The rule score is accompanied by reasons, and a person can correct the result. Gmail's own Spam label stays distinct from Maildesk's local review queue. The bounded 18-message sample makes the classroom demonstration predictable and manageable.

A separate offline demo provides fictional messages for presentation and interface review. The live path requires a correctly configured Desktop OAuth client and local MySQL server. The project clearly identifies limits such as attachments, rich text, scheduled synchronization, machine learning, and production OAuth approval as future work.

Through this project, the team can demonstrate how an object-oriented Java application communicates with an external provider, protects account boundaries in a relational schema, and reports user-visible reasons for a security-sensitive suggestion.

The completed work also establishes a practical base for future development. The current modules make it possible to add controlled pagination, improved message composition, or additional review signals without merging provider, database, and screen responsibilities into a single class. Any expansion should preserve the existing principle that mailbox owners can inspect and correct suspicious-message suggestions.

The implementation should be presented as a working academic desktop application with defined boundaries: the live Gmail path depends on local Google OAuth configuration and MySQL, while the screenshot and classroom demo path uses fictional in-memory data. This distinction is part of the project's correctness and is reflected throughout the report and its appendices.

7 - REFERENCES

1. Maildesk source repository, setup guide, architecture documentation, SQL schema, and Windows packaging notes. https://github.com/Sanjay-AI-ML/email-management-work

2. Google for Developers. Gmail API documentation. https://developers.google.com/workspace/gmail/api

3. Google Identity. OAuth 2.0 for native and desktop applications. https://developers.google.com/identity/protocols/oauth2/native-app

4. Google for Developers. Gmail API authorization scopes. https://developers.google.com/workspace/gmail/api/auth/scopes

5. Oracle. Java SE documentation. https://docs.oracle.com/en/java/

6. MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

7. Project source files: src/main/java/ems, src/main/java/com/emailapp/spam, src/demo/java, sql/schema.sql, docs/SETUP.md, and docs/ARCHITECTURE.md.

8. User-provided reference document: EMPLOYEE DETAILS AND PAYROLL MANAGEMENT SYSTEM (academic report structure used as the formatting model).

9. Oracle. Java Swing documentation. https://docs.oracle.com/javase/tutorial/uiswing/

10. Oracle. JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/basics/

11. IETF. RFC 7636: Proof Key for Code Exchange by OAuth Public Clients. https://datatracker.ietf.org/doc/html/rfc7636

12. Google for Developers. Gmail API messages resource and users.messages methods. https://developers.google.com/workspace/gmail/api/reference/rest/v1/users.messages

13. MySQL. Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/

APPENDIX A — SELECTED SOURCE CODE AND DATABASE

This appendix prints the focused source classes that demonstrate Maildesk's distinctive implementation: explainable spam scoring, the shared service boundary, bounded Gmail refresh, and transactional persistence. The full MySQL schema is included. Other project files remain available in the GitHub repository. Source lines are reproduced from the committed files; headings and short context notes are report labels.

A.1 SpamRules.java — classifier weights

```
package com.emailapp.spam;

public final class SpamRules {

    private SpamRules() {
        // Prevent object creation
    }

    public static final int BLOCKED_SENDER = 50;
    public static final int SUSPICIOUS_PHRASE = 20;
    public static final int MANY_LINKS = 15;
    public static final int URGENT_SUBJECT = 10;

    public static final int SPAM_THRESHOLD = 50;
}
```

A.2 SpamClassifier.java — explainable rule evaluation

```
package com.emailapp.spam;

import ems.model.EmailMessage;
import ems.model.SpamResult;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import java.util.regex.Pattern;

/** Explainable local rules, independent of Gmail's own SPAM classification. */
public final class SpamClassifier {
    private static final Pattern LINK = Pattern.compile("https?://\\S+", Pattern.CASE_INSENSITIVE);
    private static final List<String> PHRASES = List.of(
            "claim your prize", "verify your account", "you have won",
            "urgent action required", "click here to claim");

    public SpamResult analyze(EmailMessage message) {
        return analyze(message, false);
    }

    /** The store supplies account-specific block information within its transaction. */
    public SpamResult analyze(EmailMessage message, boolean blockedSender) {
        int score = 0;
        List<String> reasons = new ArrayList<>();
        String sender = message.getSenderEmail().trim().toLowerCase(Locale.ROOT);
        String subject = message.getSubject().toLowerCase(Locale.ROOT);
        String body = message.getBodyText().toLowerCase(Locale.ROOT);
        if (blockedSender) {
            score += SpamRules.BLOCKED_SENDER;
            reasons.add("Sender is blocked");
```

```
        }
        String content = subject + " " + body;
        int phraseMatches = 0;
        for (String phrase : PHRASES) {
            if (content.contains(phrase)) {
                score += SpamRules.SUSPICIOUS_PHRASE;
                reasons.add("Suspicious phrase: " + phrase);
                // Distinct signals accumulate; one phrase alone should not classify a message.
                if (++phraseMatches == 3) break;
            }
        }
        if (LINK.matcher(body).results().limit(5).count() >= 5) {
            score += SpamRules.MANY_LINKS;
            reasons.add("Message contains many links");
        }
        if (subject.contains("urgent") || subject.contains("act now")) {
            score += SpamRules.URGENT_SUBJECT;
            reasons.add("Urgent subject wording");
        }
        if (sender.isBlank()) reasons.add("Sender address is missing");
        return new SpamResult(score, score >= SpamRules.SPAM_THRESHOLD, reasons);
    }
}
```

A.3 SpamResult.java — classification result

```
package ems.model;

import java.util.List;

/**
 * The local detector's result. The score is a rule score, not a probability.
 * Gmail's own SPAM label remains separate from this classification.
 */
public final class SpamResult {
    private final int score;
    private final boolean suspectedSpam;
    private final List<String> reasons;

    public SpamResult(int score, boolean suspectedSpam, List<String> reasons) {
        this.score = score;
        this.suspectedSpam = suspectedSpam;
        this.reasons = List.copyOf(reasons);
    }

    public int getScore() {
        return score;
    }

    public boolean isSuspectedSpam() {
        return suspectedSpam;
    }

    public List<String> getReasons() {
        return reasons;
    }
}
```

A.4 EmailService.java — shared application contract

```
package ems.service;

import ems.exception.AppException;
import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
```

```
import ems.model.SpamResult;
import java.util.List;


/**
 * Shared boundary between the Swing interface and Gmail/database integrations.
 * Implementations support one connected account at a time and scope every
 * mailbox operation to that account. Operations can block, so the UI calls them
 * on background workers rather than the Swing event-dispatch thread.
 *
 * <p>Implementations should turn recoverable provider/database errors into
 * {@link AppException} with useful, non-sensitive messages. A failed operation
 * must not be presented to the UI as a successful update.
 */
public interface EmailService {
    /** Authorizes Gmail access through Google and returns the connected account. */
    Account connectGmail() throws AppException;


    /** Ends the application's local connected session; does not delete the Gmail account. */
    void disconnect();


    /** Fetches Gmail changes, updates the local cache, and runs local spam analysis. */
    void refresh() throws AppException;
    default void refreshRecent() throws AppException { refresh(); }


    /**
     * Returns visible cached messages, newest first, for the selected mailbox.
     * An empty query returns all messages in that view; searches match subject,
     * body, sender, or recipient addresses. Returns an empty list if none match.
     */
    List<EmailMessage> getMessages(Mailbox mailbox, String query) throws AppException;


    /**
     * Opens an accessible message for the connected account and marks it read
     * in Gmail. Returns the updated snapshot after a successful operation.
     */
    EmailMessage openMessage(String gmailMessageId) throws AppException;


    /** Sends to one real recipient address and returns Gmail's new message ID. */
    String sendMessage(String recipientEmail, String subject, String body) throws AppException;


    /** Returns the local detector's result for a message in the connected account. */
    SpamResult getSpamResult(String gmailMessageId) throws AppException;


    /** Applies Gmail's spam classification and persists the user's correction locally. */
    void markAsSpam(String gmailMessageId) throws AppException;


    /** Removes Gmail's spam classification and persists the user's correction locally. */
    void markAsNotSpam(String gmailMessageId) throws AppException;


    /** Non-sensitive notice for a successful operation with a recoverable cache issue. */
    default String getLastWarning() { return ""; }
}
```

A.5 MimeMessages.java — plain-text Gmail message construction

```
package ems.gmail;


import ems.exception.AppException;
import ems.model.EmailMessage;
import jakarta.mail.Address;
import jakarta.mail.Message;
import jakarta.mail.Multipart;
import jakarta.mail.Part;
import jakarta.mail.Session;
import jakarta.mail.internet.InternetAddress;
import jakarta.mail.internet.MimeMessage;
```

```
import java.io.ByteArrayInputStream;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.StringReader;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.ArrayList;
import java.util.Base64;
import java.util.Date;
import java.util.List;
import java.util.Properties;
import java.util.Set;
import javax.swing.text.MutableAttributeSet;
import javax.swing.text.html.HTML;
import javax.swing.text.html.HTMLEditorKit;
import javax.swing.text.html.parser.ParserDelegator;

/** RFC 822/MIME conversion. HTML is converted to inert text, never rendered or fetched. */
final class MimeMessages {
    private MimeMessages() {}

    static EmailMessage parse(long accountId, String id, String raw, Instant received,
            Set<String> labels) throws AppException {
        try {
            byte[] bytes = Base64.getUrlDecoder().decode(raw);
            MimeMessage mime = new MimeMessage(Session.getInstance(new Properties()),
                    new ByteArrayInputStream(bytes));
            Address[] from = mime.getFrom();
            String sender = from == null || from.length == 0 ? "" : address(from[0]);
            List<String> recipients = new ArrayList<>();
            Address[] allRecipients = mime.getAllRecipients();
            if (allRecipients != null) {
                for (Address recipient : allRecipients) recipients.add(address(recipient));
            }
            Date sentDate = mime.getSentDate();
            Instant date = received != null ? received
                    : sentDate == null ? Instant.EPOCH : sentDate.toInstant();
            return new EmailMessage(accountId, id, sender, recipients,
                    mime.getSubject() == null ? "" : mime.getSubject(),
                    readText(mime, 0), date, labels);
        } catch (Exception e) {
            throw new AppException("Gmail returned a message that could not be read.", e);
        }
    }

    static String compose(String sender, String recipient, String subject, String body)
            throws AppException {
        try {
            MimeMessage mime = new MimeMessage(Session.getInstance(new Properties()));
            mime.setFrom(validAddress(sender));
            mime.setRecipient(Message.RecipientType.TO, validAddress(recipient));
            if (subject == null || subject.contains("\r") || subject.contains("\n")) {
                throw new AppException("The subject must be a single line.");
            }
            mime.setSubject(subject, StandardCharsets.UTF_8.name());
            mime.setText(body == null ? "" : body, StandardCharsets.UTF_8.name());
            mime.setSentDate(new Date());
            mime.saveChanges();
            ByteArrayOutputStream bytes = new ByteArrayOutputStream();
            mime.writeTo(bytes);
            return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes.toByteArray());
        } catch (AppException e) {
            throw e;
        } catch (Exception e) {
            throw new AppException("Enter a valid sender and recipient email address.", e);
        }
    }

    private static InternetAddress validAddress(String value) throws Exception {
        if (value == null || value.contains("\r") || value.contains("\n")) {
            throw new AppException("Enter one valid recipient email address.");
        }
```

```
        InternetAddress[] addresses = InternetAddress.parse(value, true);
        if (addresses.length != 1) throw new AppException("Enter one recipient email address.");
        addresses[0].validate();
        return addresses[0];
    }

    private static String address(Address address) {
        return address instanceof InternetAddress internet ? internet.getAddress() : address.toString();
    }

    private static String readText(Part part, int depth) throws Exception {
        if (depth > 30 || Part.ATTACHMENT.equalsIgnoreCase(part.getDisposition())
                || (part.getFileName() != null && !part.getFileName().isBlank())) return "";
        if (part.isMimeType("text/plain")) return contentString(part);
        if (part.isMimeType("text/html")) return htmlToText(contentString(part));
        if (part.isMimeType("multipart/*")) {
            Multipart multi = (Multipart) part.getContent();
            // Prefer the text/plain alternative instead of displaying duplicate HTML text.
            if (part.isMimeType("multipart/alternative")) {
                String fallback = "";
                for (int i = 0; i < multi.getCount(); i++) {
                    Part child = multi.getBodyPart(i);
                    String text = readText(child, depth + 1);
                    if (child.isMimeType("text/plain") && !text.isBlank()) return text;
                    if (!text.isBlank()) fallback = text;
                }
                return fallback;
            }
            List<String> pieces = new ArrayList<>();
            for (int i = 0; i < multi.getCount(); i++) {
                String text = readText(multi.getBodyPart(i), depth + 1);
                if (!text.isBlank()) pieces.add(text);
            }
            return String.join("\n\n", pieces);
        }
        return "";
    }

    private static String contentString(Part part) throws Exception {
        Object content = part.getContent();
        return content instanceof String text ? text : "";
    }

    private static String htmlToText(String html) throws IOException {
        StringBuilder text = new StringBuilder();
        new ParserDelegator().parse(new StringReader(html), new HTMLEditorKit.ParserCallback() {
            private int hiddenDepth;
            @Override public void handleStartTag(HTML.Tag tag, MutableAttributeSet attrs, int pos) {
                if (tag == HTML.Tag.SCRIPT || tag == HTML.Tag.STYLE) hiddenDepth++;
                if (hiddenDepth == 0 && tag.isBlock()) newline();
            }
            @Override public void handleEndTag(HTML.Tag tag, int pos) {
                if (tag == HTML.Tag.SCRIPT || tag == HTML.Tag.STYLE) {
                    hiddenDepth = Math.max(0, hiddenDepth - 1);
                }
                if (hiddenDepth == 0 && tag.isBlock()) newline();
            }
            @Override public void handleSimpleTag(HTML.Tag tag, MutableAttributeSet attrs, int pos) {
                if (tag == HTML.Tag.BR) newline();
            }
            @Override public void handleText(char[] data, int pos) {
                if (hiddenDepth == 0) text.append(data);
            }
            private void newline() {
                if (text.length() > 0 && text.charAt(text.length() - 1) != '\n') text.append('\n');
            }
        }, true);
        return text.toString().strip();
    }
}
```

A.6 GmailEmailService.java — bounded refresh

Selected complete method excerpt from src/main/java/ems/service/GmailEmailService.java.

```
    @Override public synchronized void refresh() throws AppException {
        Account current = connected(); lastWarning = "";
        // A failed network fetch must never be mistaken for an empty mailbox.
        List<EmailMessage> snapshot = gmail.fetchRecentMessages(current.getAccountId(), 6)
                .stream().limit(18).toList();
        verifyAccount(current, snapshot);
        try { store.replaceSnapshot(current.getAccountId(), snapshot); }
        catch (SQLException exception) {
            throw databaseError("Gmail was read, but the local mailbox could not be updated. Check MySQL and
refresh again.", exception);
        }
    }
```

A.7 GmailAuth.java — PKCE desktop authorization

Selected complete method excerpt from src/main/java/ems/gmail/GmailAuth.java.

```
    private void authorizeInBrowser() throws AppException {
        HttpServer server = null;
        try {
            String state = randomToken();
            String verifier = randomToken();
            String challenge = Base64.getUrlEncoder().withoutPadding().encodeToString(
                    MessageDigest.getInstance("SHA-256").digest(verifier.getBytes(StandardCharsets.US_ASCII)));
            server = HttpServer.create(new InetSocketAddress("127.0.0.1", 0), 0);
            URI redirect = URI.create("http://127.0.0.1:" + server.getAddress().getPort() + "/oauth2callback");
            CompletableFuture<String> code = new CompletableFuture<>();
            server.createContext("/oauth2callback", exchange -> handleCallback(exchange, state, code));
            server.start();
            Map<String, String> form = new LinkedHashMap<>();
            form.put("client_id", clientId);
            form.put("redirect_uri", redirect.toString());
            form.put("response_type", "code");
            form.put("scope", SCOPE);
            form.put("access_type", "offline");
            form.put("prompt", "consent select_account");
            form.put("state", state);
            form.put("code_challenge", challenge);
            form.put("code_challenge_method", "S256");
            browser.open(URI.create(authorizeEndpoint + "?" + formEncode(form)));
            String authCode = code.get(timeout.toMillis(), TimeUnit.MILLISECONDS);
            Map<String, String> tokenForm = clientFields();
            tokenForm.put("grant_type", "authorization_code");
            tokenForm.put("code", authCode);
            tokenForm.put("redirect_uri", redirect.toString());
            tokenForm.put("code_verifier", verifier);
            applyTokens(exchangeToken(tokenForm));
        } catch (AppException e) {
            throw e;
        } catch (TimeoutException e) {
            throw new AppException("Google sign-in timed out. Click Connect Gmail and complete sign-in within
three minutes.", e);
        } catch (ExecutionException e) {
            if (e.getCause() instanceof AppException app) throw app;
            throw new AppException("Google sign-in could not be completed. Please try again.", e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new AppException("Google sign-in was interrupted.", e);
        } catch (Exception e) {
            throw new AppException("Could not open Google sign-in. Check that a default web browser is available
"
                    + "and local loopback connections are allowed.", e);
        } finally {
            if (server != null) server.stop(0);
        }
```

```
    }
```

A.8 MailStore.java — persistence boundary

```
package ems.storage;

import ems.model.Account;
import ems.model.EmailMessage;
import ems.model.Mailbox;
import ems.model.SpamResult;
import java.sql.SQLException;
import java.util.Collection;
import java.util.List;
import java.util.Optional;

/** Account-scoped local mailbox cache. Gmail is the source of mailbox truth. */
public interface MailStore {
    void initialize() throws SQLException;
    Account findOrCreateAccount(String email) throws SQLException;
    void saveMessages(long accountId, List<EmailMessage> messages) throws SQLException;
    void replaceSnapshot(long accountId, List<EmailMessage> messages) throws SQLException;
    List<EmailMessage> getMessages(long accountId, Mailbox mailbox, String query) throws SQLException;
    Optional<EmailMessage> findMessage(long accountId, String gmailMessageId) throws SQLException;
    SpamResult getSpamResult(long accountId, String gmailMessageId) throws SQLException;
    void saveFeedback(long accountId, String gmailMessageId, boolean spam) throws SQLException;
    void deleteMessages(long accountId, Collection<String> gmailMessageIds) throws SQLException;
}
```

A.9 JdbcMailStore.java — account-scoped snapshot replacement

Selected complete method excerpt from src/main/java/ems/storage/JdbcMailStore.java.

```
    public void replaceSnapshot(long accountId, List<EmailMessage> messages) throws SQLException {
        checkAccount(accountId, messages);
        transaction(connection -> {
            saveBatch(connection, accountId, messages);
            Set<String> present = new LinkedHashSet<>();
            for (EmailMessage message : messages) present.add(message.getGmailMessageId());
            List<String> removed = new ArrayList<>();
            try (PreparedStatement query = connection.prepareStatement("SELECT gmail_message_id FROM messages
WHERE account_id=?")) {
                query.setLong(1, accountId);
                try (ResultSet rows = query.executeQuery()) {
                    while (rows.next()) if (!present.contains(rows.getString(1)))
removed.add(rows.getString(1));
                }
            }
            deleteBatch(connection, accountId, removed);
        });
    }

    private static void checkAccount(long accountId, List<EmailMessage> messages) throws SQLException {
        for (EmailMessage message : messages) {
            if (message.getAccountId() != accountId) throw new SQLException("Message belongs to another
account.");
        }
    }
```

A.10 JdbcMailStore.java — transaction and rollback

Selected complete method excerpt from src/main/java/ems/storage/JdbcMailStore.java.

```
    private void transaction(Transaction action) throws SQLException {
```

```
        try (Connection connection = connections.open()) {
            connection.setAutoCommit(false);
            try {
                action.run(connection);
                connection.commit();
            } catch (SQLException | RuntimeException exception) {
                try { connection.rollback(); } catch (SQLException rollback) {
exception.addSuppressed(rollback); }
                throw exception;
            }
        }
    }
```

A.11 schema.sql — complete MySQL database definition

```
-- Email Management: MySQL 8.x cache schema. No OAuth tokens or passwords are stored here.
-- Use a dedicated database user with permission to create and read/write these tables.
CREATE DATABASE IF NOT EXISTS email_management CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE email_management;

CREATE TABLE IF NOT EXISTS accounts (
            account_id BIGINT AUTO_INCREMENT PRIMARY KEY,
            email_address VARCHAR(254) NOT NULL UNIQUE
        );

CREATE TABLE IF NOT EXISTS messages (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            sender_email VARCHAR(320) NOT NULL,
            subject TEXT NOT NULL,
            body_text LONGTEXT NOT NULL,
            received_at DATETIME(6) NOT NULL,
            is_read BOOLEAN NOT NULL DEFAULT TRUE,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        );

CREATE TABLE IF NOT EXISTS message_recipients (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            recipient_email VARCHAR(320) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id, recipient_email),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        );

CREATE TABLE IF NOT EXISTS message_labels (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            gmail_label_id VARCHAR(128) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id, gmail_label_id),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        );

CREATE TABLE IF NOT EXISTS spam_results (
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            score INT NOT NULL,
            suspected_spam BOOLEAN NOT NULL,
            reasons TEXT NOT NULL,
            analyzed_at DATETIME(6) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id, gmail_message_id) REFERENCES messages(account_id, gmail_message_id) ON
DELETE CASCADE
        );

CREATE TABLE IF NOT EXISTS spam_feedback (
```

```
            account_id BIGINT NOT NULL,
            gmail_message_id VARCHAR(128) NOT NULL,
            user_marked_spam BOOLEAN NOT NULL,
            corrected_at DATETIME(6) NOT NULL,
            PRIMARY KEY (account_id, gmail_message_id),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        );

CREATE TABLE IF NOT EXISTS blocked_senders (
            account_id BIGINT NOT NULL,
            sender_email VARCHAR(320) NOT NULL,
            PRIMARY KEY (account_id, sender_email),
            FOREIGN KEY (account_id) REFERENCES accounts(account_id)
        );
```

APPENDIX B — APPLICATION OUTPUT

The following outputs were captured from Maildesk's offline demonstration mode. All people, addresses, subjects, message bodies, and dates shown are fictional. The demo does not connect to Gmail or MySQL, use the network, save messages to disk, or deliver real email. These pages document the appearance and behavior of the user interface only.

B.1 Welcome and demo entry

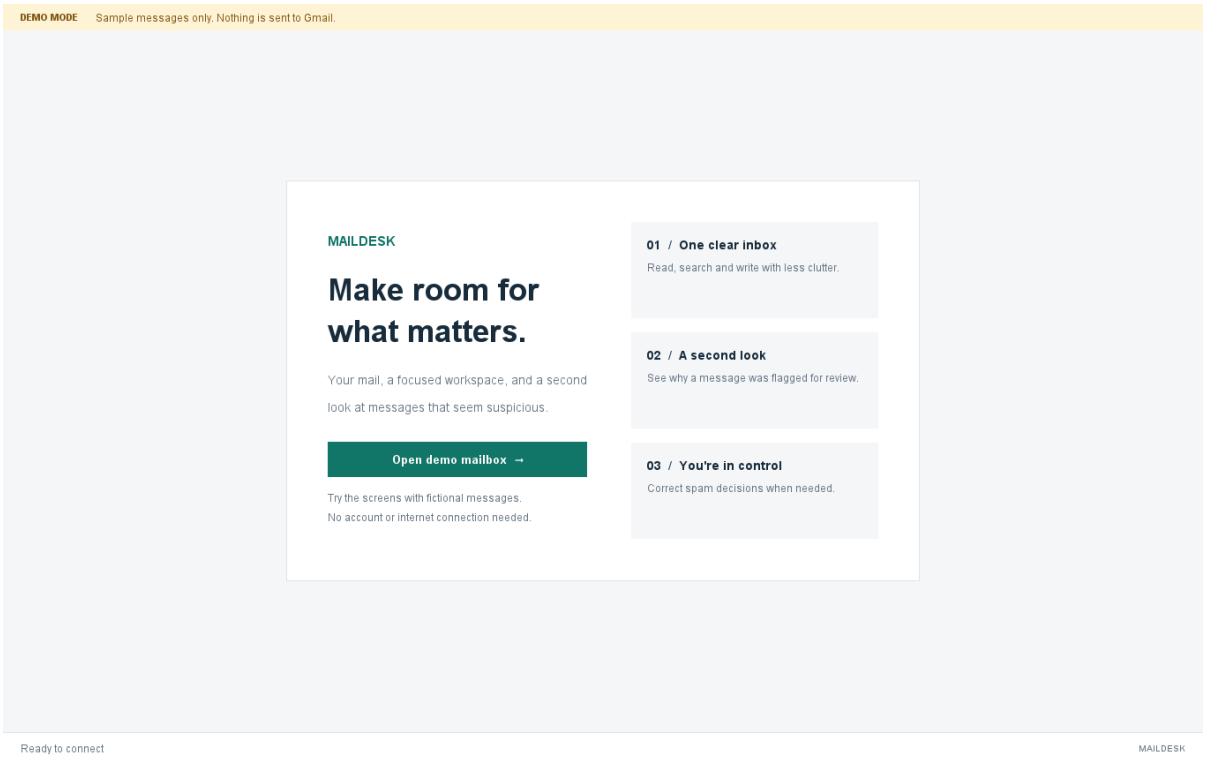


Figure 6: B.1 Welcome and demo entry

The welcome view introduces Maildesk and shows the entry point for the offline presentation. The Connect Gmail action in the live build opens Google authorization; the demo action opens fictional messages. The visible copy makes that distinction clear before the workspace is entered.

B.2 Inbox workspace

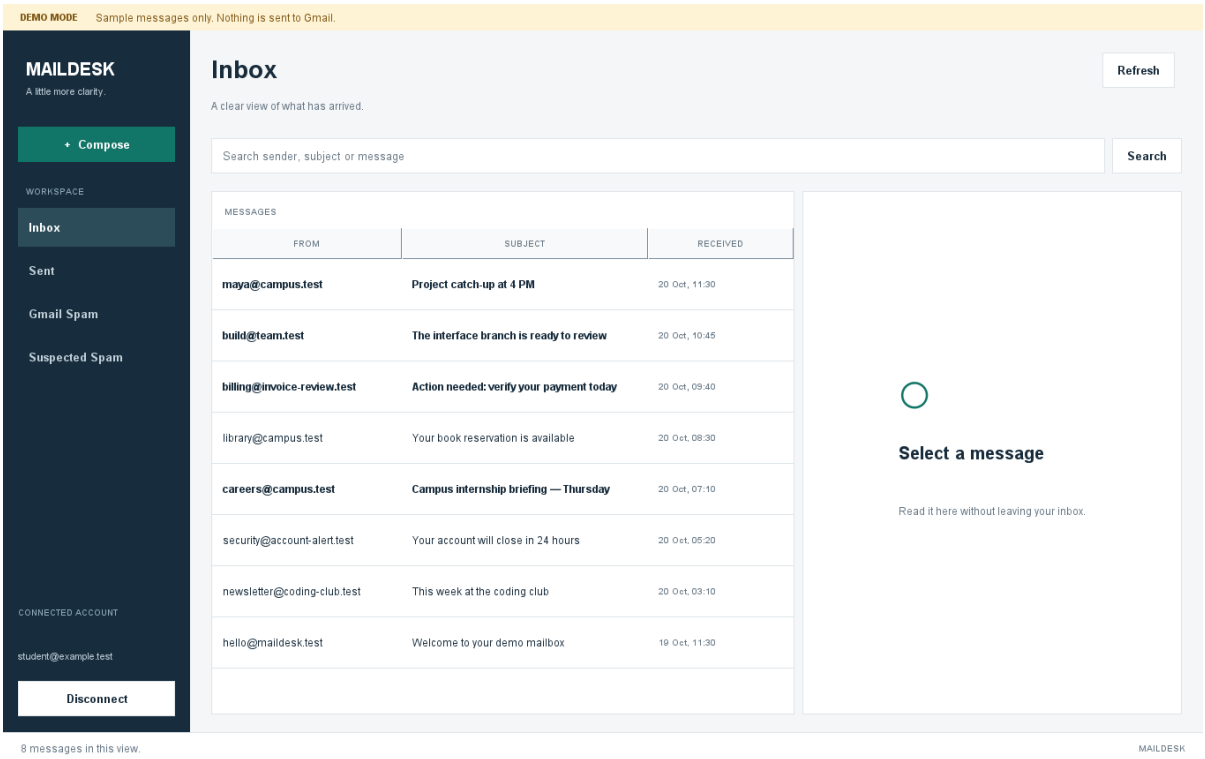


Figure 7: B.2 Inbox workspace

The workspace output shows mailbox navigation, search, the message table, selected-message context, and status controls. The rows are seeded example content. In a live session the same layout is populated from the bounded local cache retrieved through Gmail.

B.3 Message reader and explanation

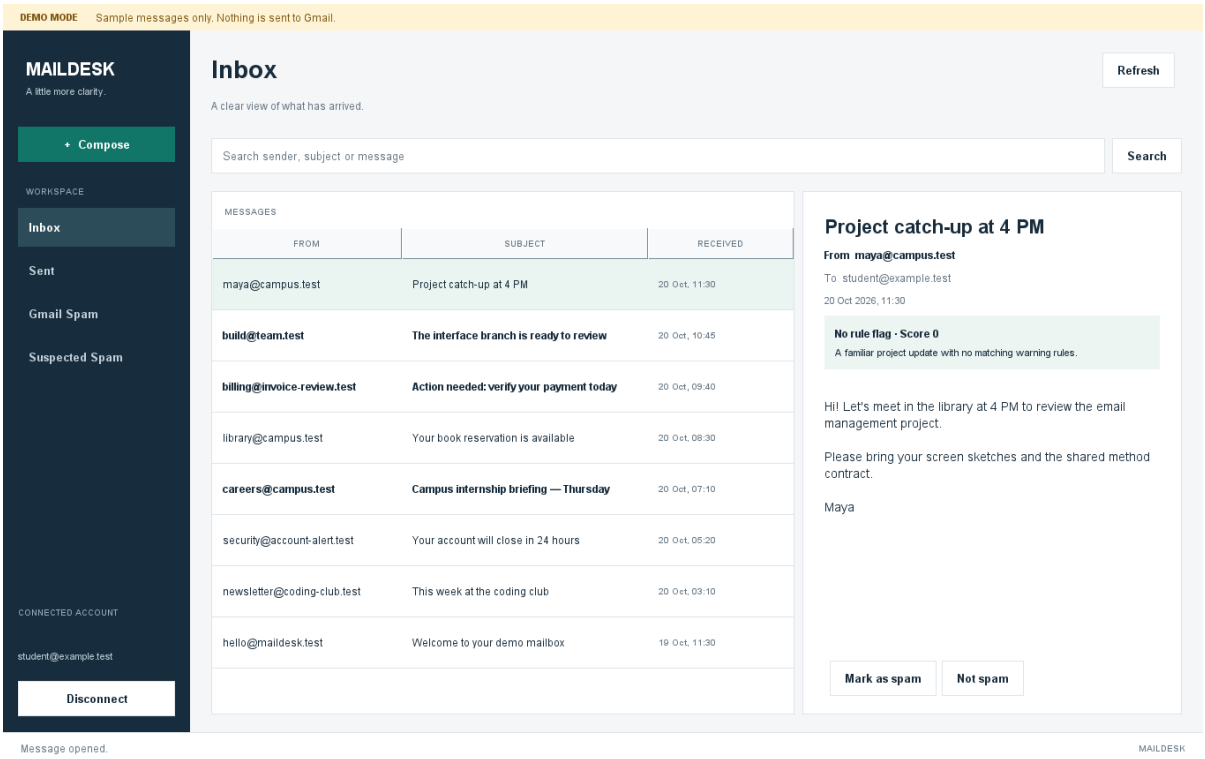


Figure 8: B.3 Message reader and explanation

The reader output shows message details and the local spam review card beside the mailbox list. Body content is displayed as plain text. A real live result depends on a successful authorized Gmail read; the image shown here is fictional demo data.

B.4 Plain-text compose form

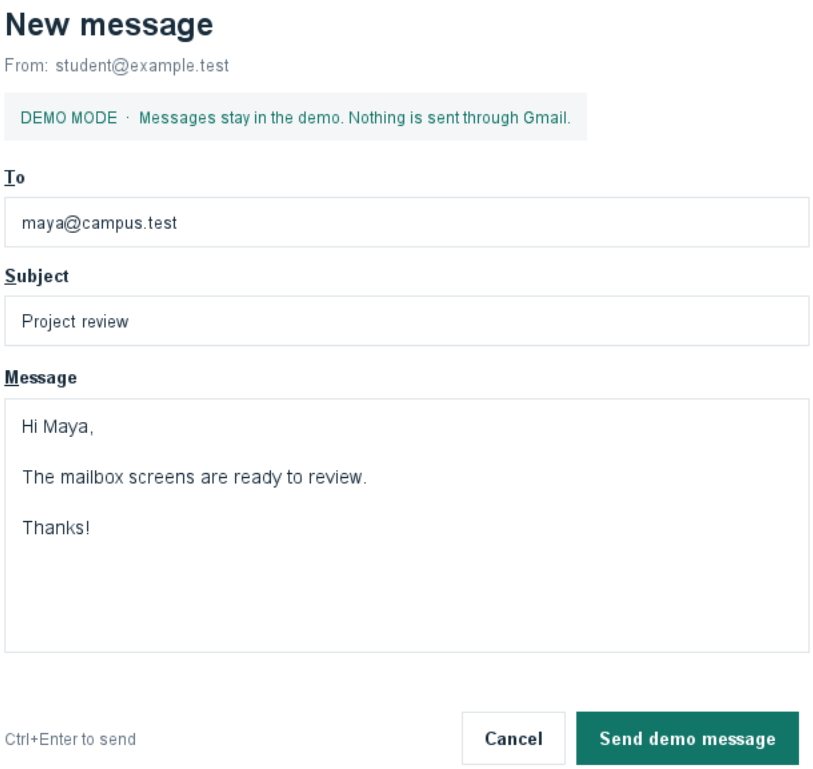


Figure 9: B.4 Plain-text compose form

The compose output shows recipient, subject, and body fields with the offline-demo notice. Validation is shared with the live path, but the demo records a simulated sent item in memory and never contacts Gmail.

B.5 Suspected Spam review

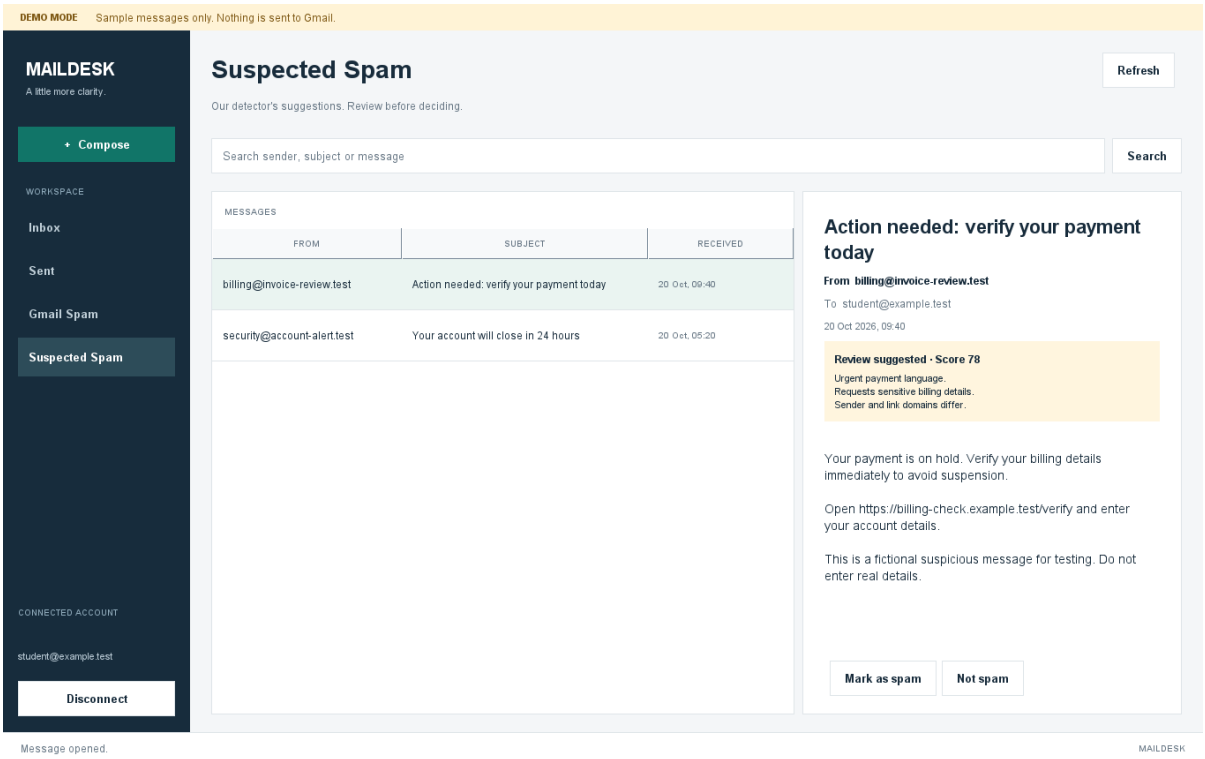


Figure 10: B.5 Suspected Spam review

This output displays a fictional suspicious message, its rule score, matched reasons, and user correction controls. It demonstrates the difference between Maildesk's local review suggestion and Gmail's own Spam label. No real message or sender is shown.